CAPÍTULO

24 Conceptos de administración de proyecto

Conceptos clave

ámbito del software 562
coordinación y comunicación. 561
descomposición de problema 563
equipo de software 558
equipos ágiles. 561
líderes de equipo. 557
participantes. 557
personal. 556
prácticas cruciales 567
principio W^{5}HH 567
producto. 562
proyecto. 566

En el prefacio de su libro acerca de la administración de proyectos de software, Meiler Page-Jones [pág. 85] hace una afirmación que pueden compartir muchos consultores de ingeniería de software:

> He visitado decenas de tiendas comerciales, tanto buenas como malas, y he observado tableros de datos que procesan los administradores, de nuevo, buenos y malos. Con mucha frecuencia, observé con horror cómo estos administradores luchan infructuosamente a través de proyectos de pesadilla, se retuercen bajo fechas límite imposibles o sistemas entregados que contrarían a sus usuarios y que se dedican a devorar enormes trozos de tiempo de mantenimiento.

Lo que Page-Jones describe son síntomas extraídos de una selección de problemas administrativos y técnicos. Sin embargo, si a los proyectos se les realizara un análisis postmortem, es muy probable que se encuentre un tema común: la administración del proyecto era débil.

En éste y en los siguientes capítulos, se presentan los conceptos clave que conducen a la administración efectiva del proyecto de software. Este capítulo considera los conceptos y principios básicos de la administración de proyectos de software. El capítulo 25 presenta las métricas de proceso y proyecto, base para la toma de decisiones administrativas efectivas. Las técnicas que se usan para estimar el costo se analizan en el capítulo 26. El capítulo 27 ayuda a definir un calendario de proyecto realista. Las actividades administrativas que conducen a mo-

Una mirada rápida

¿Qué es? Aunque muchas personas (en sus momentos más oscuros) toman la visión de Dilbert de la "administración", ésta sigue siendo una actividad muy necesaria cuando se construyen sistemas y productos basados en computadora. La administración del proyecto involucra planificación, monitoreo y control del personal, procesos y acciones que ocurren conforme el software evoluciona desde un concepto preliminar hasta su despliegue operativo completo.

¿Quién lo hace? Todo mundo "administra" en cierta medida, pero el ámbito de las actividades administrativas varía entre las personas involucradas en un proyecto de software. Un ingeniero del software administra sus actividades cotidianas, planifica, monitorea y controla las tareas técnicas. Los gerentes de proyecto planifican, monitorean y controlan el trabajo de un equipo de ingenieros de software. Los gerentes ejecutivos coordinan la interfaz entre la empresa y los profesionales del software.

¿Por qué es importante? Construir software de computadora es una labor compleja, particularmente si involucra a muchas personas que trabajan durante un tiempo relativamente largo. Por eso es necesario administrar los proyectos de software.

¿Cuáles son los pasos? Comprender las cuatro P: personal, producto, proceso y proyecto. El personal debe organizarse para realizar el trabajo de software de manera efectiva. La comunicación con el cliente y con otros participantes debe ocurrir de modo que el ámbito del producto y los requerimientos sean comprensibles. Debe seleccionarse un proceso que sea adecuado para el personal y el producto. El proyecto debe planificarse, estimándose el esfuerzo y el cronograma necesarios para concluir las tareas: definición de los productos operativos, establecimiento de los puntos de verificación de calidad, identificación de mecanismos para monitorear y control del trabajo definido por el plan.

¿Cuál es el producto final? En cuanto inician las actividades de administración, se produce un plan de proyecto que define el proceso y las tareas que se van a realizar, el personal que hará el trabajo y los mecanismos que se emplearán para valorar riesgos, controlar el cambio y evaluar la calidad.

¿Cómo me aseguro de que lo hice bien? Nunca se está completamente seguro de que el plan del proyecto es correcto, hasta que se entrega un producto de alta calidad a tiempo y dentro del presupuesto. Sin embargo, un gerente de proyecto lo hace bien cuando alienta al personal del software a trabajar en conjunto, como un equipo efectivo, y cuando enfoca su atención en las necesidades del cliente y en la calidad del producto.

nitoreo, mitigación y administración efectiva del riesgo se presentan en el capítulo 28. Finalmente, el capítulo 29 considera el mantenimiento y la reingeniería, y estudia los conflictos administrativos que se encontrarán cuando lidie con sistemas heredados.

24.1 El espectro administrativo

La administración efectiva de un proyecto de software se enfoca en las cuatro P: personal, producto, proceso y proyecto. El orden no es arbitrario. El gerente que olvida que el trabajo de la ingeniería del software es una empresa intensamente humana nunca triunfará en la administración del proyecto. Un gerente que fracase en alentar una comunicación comprensiva con los participantes durante las primeras etapas de la evolución de un producto se arriesga a construir una solución elegante para el problema equivocado. El gerente que ponga poca atención al proceso corre el riesgo de insertar métodos y herramientas técnicos competentes pero en el vacío. Aquel que se embarque sin un plan sólido pone en peligro el éxito del proyecto.

24.1.1 El personal

Desde la década de 1960 se estudia la formación de personal de software motivado y enormemente calificado. De hecho, el "factor humano" es tan importante que el Software Engineering Institute desarrolló un *Modelo de madurez de capacidades del personal* (People-CMM, por sus siglas en inglés), en reconocimiento al hecho de que "toda organización requiere mejorar continuamente su habilidad para atraer, desarrollar, motivar, organizar y conservar la fuerza de trabajo necesaria a fin de lograr sus objetivos empresariales estratégicos" [Cur01].

El People-CMM define las siguientes áreas prácticas clave para el personal de software: plantilla, comunicación y coordinación, ambiente de trabajo, desempeño administrativo, capacitación, compensación, análisis y desarrollo de competencias, desarrollo profesional, desarrollo de grupo de trabajo y desarrollo de equipo/cultura, entre otros. Las organizaciones que conforme a este modelo logran altos niveles de madurez de capacidades de personal tienen una probabilidad muy elevada de alcanzar la implementación de prácticas administrativas efectivas en los proyectos de software.

El People-CMM es un compañero de la *Integración del modelo de madurez de capacidades del software* (capítulo 30), que guía a las organizaciones en la creación de un proceso de software maduro. Los conflictos asociados con la administración del personal y con la estructura para los proyectos de software se consideran más adelante, en este capítulo.

24.1.2 El producto

Antes de poder planear un proyecto, deben establecerse los objetivos y el ámbito del producto, considerarse soluciones alternativas e identificar las restricciones técnicas y administrativas. Sin esta información, es imposible definir estimaciones razonables (y precisas) del costo, una valoración efectiva del riesgo, una descomposición realista de las tareas del proyecto y un calendario de proyecto manejable que proporcione en cada momento un indicio significativo del progreso.

Como desarrolladores de software, todos los participantes deben reunirse para definir los objetivos y el ámbito del producto. En muchos casos, esta actividad comienza como parte de la ingeniería del sistema o de la ingeniería del proceso empresarial y continúa como el primer paso en la ingeniería de requerimientos del software (capítulo 5). Los objetivos identifican las metas globales para el producto (desde el punto de vista de los participantes) sin considerar cómo se lograrán estas metas. El ámbito identifica los datos, funciones y comportamientos principales que caracterizan al producto y, más importante, intenta ligar dichas características en forma cuantitativa.

Una vez comprendidos los objetivos y el ámbito del producto, se consideran soluciones alternativas. Aunque se analizan muy pocos detalles, las alternativas permiten a los gerentes y profesionales seleccionar un "mejor" enfoque, dadas las restricciones impuestas por fechas de entrega, restricciones presupuestales, disponibilidad de personal, interfaces técnicas y muchos otros factores.

24.1.3 El proceso



Quienes se adhieren a la filosofía de proceso ágil (capítulo 3) argumentan que su proceso es más esbelto que otros. Esto puede ser cierto, pero todavía tienen un proceso, y la ingeniería de software ágil todavía requiere disciplina.

Un proceso de software (capítulos 2 y 3) proporciona el marco conceptual desde el cual puede establecerse un plan completo para el desarrollo de software. Un pequeño número de actividades de marco conceptual se aplica a todos los proyectos de software, sin importar su tamaño o complejidad. Algunos conjuntos de diferentes tareas (tareas, hitos, productos operativos y puntos de aseguramiento de calidad) permiten que las actividades del marco conceptual se adapten a las características del proyecto de software y a los requerimientos del equipo del proyecto. Finalmente, las actividades sombrilla (como el aseguramiento de la calidad del software, la administración de configuración del software y las mediciones) recubren el modelo de proceso. Las actividades sombrilla son independientes de cualquier actividad del marco conceptual y ocurren a lo largo del proceso.

24.1.4 El proyecto

Los proyectos de software se planean y controlan debido a una razón principal: es la única forma conocida para manejar la complejidad. E incluso así, los equipos de software todavía batallan. En un estudio de 250 grandes proyectos de software desarrollados entre 1998 y 2004, Capers Jones [Jon04] encontró que "alrededor de 25 se consideraron exitosos por haber logrado sus objetivos de calendario, costo y calidad. Aproximadamente 50 tuvieron demoras o excesos por abajo de 35 por ciento, mientras que más o menos 175 experimentaron grandes demoras y excesos, o se dieron por concluidos sin completarse". Aunque actualmente la tasa de éxito para los proyectos de software puede haber mejorado un poco, la tasa de falla de proyecto sigue siendo mucho más alta de lo que debiera.[1]

Para evitar el fracaso del proyecto, un gerente de proyecto de software y los ingenieros de software que construyan el producto deben evitar un conjunto de señales de advertencia comunes, entender los factores de éxito cruciales que conducen a una buena administración del proyecto y desarrollar un enfoque de sentido común para planificar, monitorear y controlar el proyecto. Cada uno de estos temas se estudia en la sección 24.5 y en los capítulos que siguen.

24.2 El personal

En un estudio publicado por el IEEE [Cur88], se preguntó a los vicepresidentes de ingeniería de tres grandes compañías tecnológicas cuál era el elemento más importante para el éxito de un proyecto de software. Ellos respondieron de la siguiente manera:

VP 1: Supongo que, si tienes que elegir una cosa que sea la más importante en nuestro ambiente, diría que no son las herramientas que usamos, es el personal.

VP 2: El ingrediente más importante que fue exitoso en este proyecto fue tener gente inteligente [...] en mi opinión, muy pocas cosas más importan [...] La cosa más importante que

1 Con estas estadísticas, es razonable preguntar cómo sigue creciendo exponencialmente el impacto de las computadoras. Parte de la respuesta es que un número sustancial de estos proyectos "fallidos" estuvieron mal concebidos desde el inicio. Los clientes pierden interés rápidamente (porque lo que pidieron en realidad no era tan importante como pensaron la primera vez) y los proyectos se cancelan.

haces para un proyecto es seleccionar al personal [...] El éxito de la organización de desarrollo de software está muy, muy asociada con la habilidad para reclutar buen personal.

VP 3: La única regla que tengo en la administración es asegurarme de que tengo buen personal, gente realmente buena, y que hago crecer gente buena y que proporciono un ambiente en el que la gente buena puede producir.

De hecho, éste es un testimonio convincente acerca de la importancia del personal en el proceso de ingeniería de software. Y aún así, para la mayoría de las personas, desde los vicepresidentes ejecutivos de ingeniería hasta el profesional en el nivel más bajo, con frecuencia dan por hecho al personal. Los administradores argumentan (como lo hizo el grupo anterior) que las personas son lo importante, pero sus acciones en ocasiones contradicen sus palabras. En esta sección se examina a las personas que participan en el proceso de software y la forma en la que se organizan para realizar ingeniería de software efectiva.

24.2.1 Los participantes

El proceso de software (y todo proyecto de software) está poblado de participantes, quienes pueden organizarse en alguna de las siguientes áreas:

1. *Gerentes ejecutivos*, quienes definen los temas empresariales que con frecuencia tienen una influencia significativa sobre el proyecto.
2. *Gerentes de proyecto (técnicos)*, quienes deben planificar, motivar, organizar y controlar a los profesionales que hacen el trabajo de software.
3. *Profesionales* que aportan las habilidades técnicas que se necesitan para someter a ingeniería un producto o aplicación.
4. *Clientes* que especifican los requerimientos para el software que se va a fabricar, así como otros participantes que tienen un interés periférico en el resultado.
5. *Usuarios finales*, quienes interactúan con el software una vez que se libera para su uso productivo.

Todo proyecto de software está poblado con personas que están dentro de esta taxonomía.[2] Para ser efectivo, el equipo de software debe organizarse de manera que maximice las habilidades y capacidades de cada persona. Y ésta es labor del líder del equipo.

24.2.2 Líderes de equipo

La administración del proyecto es una actividad que implica mucho trato con la gente; por esta razón, los profesionales competentes tienen con frecuencia pobre desempeño como líderes de equipo. Simplemente, no tienen la mezcla justa de habilidades personales. Y aún así, como Edgemon afirma: "Por desgracia, y por muy frecuente que parezca, los individuos simplemente se topan con el papel de gerente de proyecto y se convierten en gerentes accidentales de proyecto" [Edg95].

En un excelente libro acerca del liderazgo técnico, Jerry Weinberg [Wei86] sugiere un modelo MOI de liderazgo:

¿Qué se busca cuando se elige a alguien como líder de un proyecto de software?

Motivación. Habilidad para alentar (mediante "empuje o jalón") al personal técnico a producir a su máxima capacidad.

Organización. Habilidad para moldear los procesos existentes (o inventar nuevos) que permitirán que el concepto inicial se traduzca en un producto final.

2 Cuando se desarrollan *webapps*, personal no técnico puede involucrarse en la creación de contenido.

Ideas o innovación. Habilidad para alentar a las personas a crear y sentirse creativas, aun cuando deban trabajar dentro de fronteras establecidas para un producto o aplicación de software particular.

Weinberg sugiere que los líderes de proyecto exitosos aplican un estilo administrativo de resolución de problemas. Es decir, un gerente de proyecto de software debe concentrarse en comprender el problema que se va a resolver, en administrar el flujo de ideas y, al mismo tiempo, en dejar que todos en el equipo sepan (por medio de palabras y, mucho más importante, con acciones) que la calidad cuenta y que no se comprometerá.

Cita:

"En términos más simples, un líder es aquel que sabe a dónde quiere ir, y se levanta y marcha."

John Erskine

Otra visión [Edg95] de las características que definen a un gerente de proyecto eficaz enfatiza cuatro rasgos clave:

Resolución de problemas. Un gerente de proyecto de software eficaz puede diagnosticar los conflictos técnicos y organizativos que son más relevantes, estructura sistemáticamente una solución o motiva adecuadamente a otros profesionales para desarrollarla, aplica lecciones aprendidas de proyectos pasados a situaciones nuevas y sigue siendo suficientemente flexible para cambiar de dirección si los intentos por resolver el problema son infructuosos.

Identidad administrativa. Un buen gerente de proyecto debe hacerse cargo del mismo. Debe tener la confianza para asumir el control cuando sea necesario y asegurarse de permitir que el buen personal técnico siga sus instintos.

Logro. Un gerente competente debe recompensar la iniciativa y el logro para optimizar la productividad de un equipo de proyecto. Debe demostrar mediante sus acciones que no se castigará del correr riesgos de manera controlada.

Influencia y construcción del equipo. Un gerente de proyecto eficaz debe poder "leer" a la gente; debe poder comprender las señales verbales y no verbales, y reaccionar ante las necesidades de las personas que envían estas señales. El gerente debe permanecer bajo control en situaciones de alto estrés.

24.2.3 El equipo de software

Cita:

"No todo grupo es un equipo y no todo equipo es eficaz."

Glenn Parker

Existen casi tantas estructuras organizativas humanas para el desarrollo del software como organizaciones que lo desarrollan. Para bien o para mal, la estructura organizativa no puede modificarse fácilmente. La preocupación por las consecuencias prácticas y por las políticas del cambio organizativo no está dentro del ámbito de responsabilidad del gerente del proyecto de software. Sin embargo, la organización de las personas directamente involucradas en un nuevo proyecto de software está dentro del campo de acción del gerente del proyecto.

La "mejor" estructura de equipo depende del estilo administrativo de la organización, del número de personas que formarán el equipo y de sus niveles de habilidad, así como de la dificultad global del problema. Mantei [Man81] describe siete factores de proyecto que deben considerarse cuando se planee la estructura de los equipos de ingeniería de software:

? ¿Qué factores deben considerarse cuando se elige la estructura de un equipo de software?

- Dificultad del problema que se va a resolver.
- "Tamaño" del programa resultante en líneas de código o puntos de función.
- Tiempo que el equipo permanecerá unido (vida del equipo).
- Grado en el que puede dividirse en módulos el problema.
- Calidad y confiabilidad requeridas por el sistema que se va a construir.
- Rigidez de la fecha de entrega.
- Grado de sociabilidad (comunicación) requerido para el proyecto.

Constantine [Con93] sugiere cuatro "paradigmas organizacionales" para los equipos de ingeniería de software:

¿Qué opciones se tienen cuando se define la estructura de un equipo de software?

1. Un *paradigma cerrado* estructura un equipo conforme a una jerarquía de autoridad tradicional. Tales equipos pueden trabajar bien cuando producen software muy similar al de esfuerzos anteriores, pero será menos probable que sean innovadores cuando trabajen dentro de este paradigma.
2. Un *paradigma aleatorio* estructura un equipo de manera holgada y depende de la iniciativa individual de los miembros del equipo. Cuando se requiere innovación o avance tecnológico, destacarán los equipos que siguen este paradigma, pero pueden batallar cuando se requiera "desempeño ordenado".
3. Un *paradigma abierto* intenta estructurar un equipo de manera que logre algunos de los controles asociados con el paradigma cerrado, pero también mucha de la innovación que ocurre cuando se usa el paradigma aleatorio. El trabajo se realiza de manera colaboradora; la gran comunicación y la toma de decisiones consensuadas constituyen las características de los equipos de paradigma abierto. Las estructuras de equipo de este paradigma son muy adecuadas para la solución de problemas complejos, pero pueden no desempeñarse tan eficazmente como otros equipos.
4. Un *paradigma síncrono* se apoya en la compartimentalización natural de un problema y organiza a los miembros del equipo para trabajar en trozos del problema con poca comunicación activa entre ellos.

Cita:

"Si quieres ser incrementalmente mejor: sé competitivo. Si quieres ser exponencialmente mejor: sé cooperativo."

Autor desconocido

Como acotación histórica, cabe decir que una de las primeras organizaciones de equipo de software fue una estructura de paradigma cerrado originalmente llamado *equipo de programador jefe*. Esta estructura la propuso por primera ocasión Harlan Mills y la describió Baker [Bak72]. El núcleo del equipo estaba compuesto de: un *ingeniero ejecutivo* (el programador jefe), quien planeaba, coordinaba y revisaba todas las actividades técnicas del equipo; *personal técnico* (por lo general de dos a cinco personas), quienes realizaban análisis y desarrollaban actividades; y un *ingeniero de respaldo*, quien apoyaba al ingeniero ejecutivo en sus actividades y podía sustituirlo con mínima pérdida en la continuidad del proyecto. El programador jefe puede auxiliarse con uno o más especialistas (por ejemplo, experto en telecomunicaciones, diseñador de bases de datos), personal de apoyo (por ejemplo, escritores técnicos, oficinistas) y un bibliotecario de software.

Como contrapunto a la estructura del equipo del programador jefe, el paradigma aleatorio de Constantine [Con93] sugiere un equipo de software con independencia creativa cuyo enfoque para trabajar pueda denominarse de mejor manera como *anarquía innovadora*. Aunque el enfoque de espíritu libre en el trabajo de software es atractivo, canalizar la energía creativa hacia un equipo de alto rendimiento debe ser una meta central de una organización de ingeniería de software. Para lograr un equipo de alto rendimiento:

- Los miembros del equipo deben tenerse confianza entre sí.
- La distribución de habilidades debe ser adecuada para el problema.
- Es posible que tenga que excluirse del equipo a los inconformes si debe mantenerse la cohesión del equipo.

Sin importar el tipo de organización del equipo, el objetivo para todo gerente de proyecto es ayudar a crear un equipo que muestre cohesión. En su libro *Peopleware*, DeMarco y Lister [DeM98] analizan este tema:

> Tendemos a usar la palabra equipo con mucha holgura en el mundo empresarial, y llamamos así a cualquier grupo de personas asignadas para trabajar juntas. Pero muchos de estos grupos simple-

¿Qué es un equipo "cuajado"?

mente no parecen equipos. No tienen una definición común de éxito o algún espíritu de equipo identificable. Lo que falta es un fenómeno que llamamos *cuajar*.

Un equipo cuajado es un grupo de personas tan fuertemente unido que el todo es mayor que la suma de las partes [...].

Una vez que un equipo comienza a cuajarse, la probabilidad de éxito va hacia arriba. El equipo puede volverse imparable, una fuerza arrasadora para el éxito [...] No necesita ser administrado en la forma tradicional y, ciertamente, no necesita ser motivado. Adquiere cantidad de movimiento.

DeMarco y Lister sostienen que los miembros de los equipos cuajados son significativamente más productivos y más motivados que el promedio. Comparten una meta común, una cultura común y en muchos casos un "sentido de élite" que los hace únicos.

¿Por qué fallan los equipos cuando deben cuajar?

Pero no todos los equipos cuajan. De hecho, muchos equipos sufren de lo que Jackman [Jac98] llama "toxicidad de equipo". Ella define cinco factores que "fomentan un ambiente de equipo potencialmente tóxico": 1) una atmósfera de trabajo frenético, 2) alta frustración que causa fricción entre los miembros del equipo, 3) un proceso de software "fragmentado o pobremente coordinado", 4) una definición poco clara de los roles en el equipo de software y 5) "continua y repetida exposición al fracaso".

Cita:

"Hacer o no hacer. No hay intento."

Yoda, de *La guerra de las galaxias*

Para evitar un ambiente de trabajo frenético, el gerente del proyecto debe estar seguro de que el equipo tiene acceso a toda la información requerida para hacer el trabajo y de que las metas y objetivos principales, una vez definidos, no deben modificarse a menos que sea absolutamente necesario. Un equipo de software puede evitar la frustración si se le da tanta responsabilidad para la toma de decisiones como sea posible. Un proceso inadecuado (por ejemplo, tareas innecesarias o abrumadoras o productos operativos pobremente elegidos) puede evitarse al entender el producto que se va a construir y a las personas que hacen el trabajo, así como al permitir al equipo seleccionar el modelo de proceso. El equipo mismo debe establecer sus propios mecanismos de responsabilidad (las revisiones técnicas[3] son una excelente forma de lograr esto) y definir una serie de enfoques correctos cuando un miembro del equipo tiene fallos en el desempeño. Finalmente, la clave para evitar una atmósfera de fracaso radica en establecer técnicas basadas en equipo para retroalimentarse y resolver problemas.

Además de las cinco toxinas descritas por Jackman, un equipo de software con frecuencia batalla con los diferentes rasgos humanos de sus miembros. Algunos son extrovertidos; otros, introvertidos. Algunas personas reúnen información de manera intuitiva y separan los conceptos abarcadores de los hechos dispares. Otras procesan la información de manera lineal, y reúnen y organizan detalles minúsculos de los datos proporcionados. Ciertos miembros del equipo se sienten cómodos al tomar decisiones sólo cuando se presenta un argumento lógico y ordenado. Otros son intuitivos y quieren tomar decisiones con base en "corazonadas". Algunos profesionales quieren calendarios detallados poblados de tareas organizadas que les permitan lograr el cierre para algún elemento de un proyecto. Otros prefieren un ambiente más espontáneo en el que los temas abiertos sean bien vistos. Algunos trabajan duro para hacer que las cosas estén listas mucho antes de una fecha final y, por tanto, evitan el estrés conforme la fecha se aproxima, mientras que otros se sienten energizados por la adrenalina que produce una entrega de último minuto. Una discusión detallada de la psicología de estos rasgos y de las formas en las que el líder del equipo habilidoso puede ayudar a las personas con rasgos opuestos para trabajar en conjunto está más allá del ámbito de este libro.[4] Sin embargo, es importante observar que el reconocimiento de las diferencias humanas es el primer paso hacia la creación de equipos que cuajen.

3 Las revisiones técnicas se estudian con detalle en el capítulo 15.

4 Una excelente introducción a estos temas, en su relación con los equipos de proyecto de software, puede encontrarse en [Fer98].

24.2.4 Equipos ágiles

Durante las décadas pasadas, el desarrollo de software ágil (capítulo 3) se ha sugerido como antídoto a muchos de los problemas que plagan el trabajo en un proyecto de software. Cabe recordar que la filosofía ágil alienta la satisfacción del cliente y la entrega incremental temprana del software, así como pequeños equipos de proyecto enormemente motivados, métodos informales, mínimos productos operativos de ingeniería de software y simplicidad de desarrollo global.

El pequeño equipo de trabajo enormemente motivado, también llamado *equipo ágil*, adopta la mayoría de las características de los equipos de proyecto de software exitosos que se estudiaron en la sección anterior y evita muchas de las toxinas que crean problemas. No obstante, la filosofía ágil subraya la competencia individual (miembro de equipo), acoplada con la colaboración grupal como factores de éxito vitales para el equipo. Cockburn y Highsmith [Coc01a] observan esto cuando escriben:

> Si el personal del proyecto es suficientemente bueno, puede usar casi cualquier proceso y lograr esta asignación. Si no lo es, ningún proceso reparará su inadecuación: "personal mata proceso" es una forma de decirlo. Sin embargo, la falta de apoyo de usuarios y ejecutivos puede matar al proyecto: "política mata personal". El apoyo inadecuado puede impedir que incluso el personal bueno logre esta tarea.

PUNTO CLAVE

Un equipo ágil es un equipo autoorganizado que tiene autonomía para planificar y tomar decisiones técnicas.

Para hacer uso efectivo de las competencias de cada miembro del equipo y fomentar la colaboración efectiva a través de un proyecto de software, los equipos ágiles son *autoorganizados*. Un equipo autoorganizado no necesariamente mantiene una sola estructura de equipo, sino que usa elementos de los paradigmas aleatorio, abierto y síncrono de Constantine, estudiados en la sección 24.2.3.

Muchos modelos de proceso ágil (por ejemplo, Scrum) dan al equipo ágil significativa autonomía para tomar las decisiones administrativas y técnicas del proyecto necesarias para hacer que el trabajo se cumpla. La planificación se mantiene al mínimo y al equipo se le permite seleccionar su propio enfoque (por ejemplo, proceso, métodos, herramientas), restringido únicamente por los requerimientos empresariales y los estándares de la organización. Conforme avanza el proyecto, el equipo se autoorganiza para enfocarse en la competencia individual, de manera que ésta sea más benéfica para el proyecto en un momento determinado. Para lograr esto, un equipo ágil puede realizar reuniones grupales diarias para coordinar y sincronizar el trabajo que debe realizarse en ese día.

Con base en la información obtenida durante dichas reuniones, el equipo adapta su enfoque para lograr un incremento de trabajo. Conforme transcurre cada día, la autoorganización y la colaboración continuas mueven al equipo hacia un incremento de software completo.

24.2.5 Conflictos de coordinación y comunicación

Cita:

"La propiedad colectiva no es más que una ilustración de la idea de que los productos deben atribuirse al equipo [ágil], no a los individuos que constituyen el equipo."

Jim Highsmith

Existen muchas razones por las que los proyectos de software tienen problemas. La escala de muchos esfuerzos de desarrollo es grande, lo que conduce a complejidad, confusión y dificultades significativas en la coordinación de los miembros del equipo. La incertidumbre es común, lo que da como resultado un torrente continuo de cambios que detienen al equipo de proyecto. La interoperabilidad se ha convertido en una característica clave de muchos sistemas. El software nuevo debe comunicarse con el software existente y ajustarse a las restricciones predefinidas impuestas por el sistema o por el producto.

Tales características del software moderno (escala, incertidumbre e interoperabilidad) son hechos de la vida. Para lidiar con ellos de manera efectiva, deben implantarse métodos efectivos a fin de coordinar al personal que hace el trabajo. Esto se logra estableciendo mecanismos para la comunicación formal e informal entre los miembros del equipo y entre los distintos equipos. La comunicación formal se consigue mediante "comunicación escrita, reuniones estructuradas y otros canales de comunicación relativamente no interactivos e impersonales" [Kra95]. La co-

municación informal es más personal. Los miembros de un equipo de software comparten ideas sobre una base *ad hoc*, piden ayuda cuando surgen problemas e interactúan unos con otros diariamente.

CASASEGURA



Estructura del equipo

La escena: Oficina de Doug Miller antes de iniciar el proyecto de software *CasaSegura*.

Personajes: Doug Miller (gerente del equipo de ingeniería de software *CasaSegura*) y Vinod Raman, Jamie Lazar y otros miembros del equipo de ingeniería de software del producto.

La conversación:

Doug: ¿Han tenido oportunidad de consultar la información preliminar que preparó mercadotecnia acerca de *CasaSegura*?

Vinod (asiente y observa a sus compañeros de equipo): Sí. Pero tenemos muchas preguntas.

Doug: Dejemos eso por un momento. Me gustaría hablar acerca de cómo vamos a estructurar el equipo, quién es responsable de qué...

Jamie: Yo estoy totalmente a favor de la filosofía ágil, Doug. Creo que debemos ser un equipo autoorganizado.

Vinod: Estoy de acuerdo. Dada la apretada línea de tiempo y algo de la incertidumbre, así como el hecho de que todos somos realmente competentes [risas], ésta parece ser la forma correcta de avanzar.

Doug: Está bien por mí, pero ustedes conocen las instrucciones.

Jamie (sonríe y habla como si recitara algo): "Tomamos decisiones tácticas acerca de quién hace qué y cuándo, pero es nuestra responsabilidad sacar el producto a tiempo."

Vinod: Y con calidad.

Doug: Exactamente. Pero recuerden que hay restricciones. Mercadotecnia define los incrementos del software que se va a producir... consultándonos, desde luego.

Jamie: ¿Y?

Doug: Y vamos a usar UML como nuestro enfoque de modelado.

Vinod: Pero mantendremos la documentación extraña en un mínimo absoluto.

Doug: ¿Quién es el enlace conmigo?

Jamie: Decidimos que Vinod sea el líder técnico; tiene más experiencia, así que Vinod es tu enlace, pero siéntete en libertad de hablar con cualquiera de nosotros.

Doug (ríe): No se preocupen. Lo haré.

24.3 EL PRODUCTO

Un gerente de proyecto de software se enfrenta con un dilema en el comienzo mismo de un proyecto de software. Se requieren estimaciones cuantitativas y un plan organizado, pero no hay información sólida disponible. Un análisis detallado de los requerimientos del software proporcionaría la información necesaria para las estimaciones, pero el análisis usualmente tarda semanas o incluso meses en completarse. Peor aún, los requerimientos pueden ser fluidos y cambiar con regularidad conforme avanza el proyecto. Y, sin embargo, ¡se necesita un plan "ahora"!

Le guste o no al gerente de proyecto, debe examinar el producto; se pretende que el problema se resuelva desde el principio mismo del proyecto; cuando menos, debe establecer y acotar el ámbito del producto.

24.3.1 Ámbito del software

La primera actividad en la administración del proyecto de software es determinar el *ámbito del software*, que se define al responder las siguientes preguntas:



Si no puede acotar una característica del software que intenta construir, mencione la característica como un riesgo del proyecto (capítulo 25).

Contexto. ¿Cómo encaja en un sistema, producto o contexto empresarial más grande el software que se va a construir y qué restricciones se imponen como resultado del contexto?

Objetivos de información. ¿Qué objetos de datos visibles para el cliente se producen como salida del software? ¿Qué objetos de datos se requieren como entrada?

Función y desempeño. ¿Qué función realiza el software para transformar los datos de entrada en salida? ¿Existe alguna característica de desempeño especial que deba abordarse?

El ámbito del proyecto de software no debe tener ambigüedades ni ser incomprensible en los niveles administrativo y técnico. Debe acotar un enunciado del ámbito del software; es decir, los datos cuantitativos (por ejemplo, número de usuarios simultáneos, entorno objetivo, máximo tiempo de respuesta permisible) se enuncian de manera explícita, se anotan las restricciones y/o limitaciones (por ejemplo, el costo del producto restringe el tamaño de la memoria) y se describen los factores mitigantes (por ejemplo, los algoritmos deseados están bien entendidos y disponibles en Java).

24.3.2 Descomposición del problema



Para desarrollar un plan de proyecto razonable, debe descomponer el problema. Esto puede lograrse usando una lista de funciones o con casos de uso.

La descomposición del problema, en ocasiones llamada *división* o *elaboración del problema*, es una actividad que se asienta en el centro del análisis de requerimientos del software (capítulos 6 y 7). Durante la actividad de determinación del ámbito, no se hacen intentos por descomponer completamente el problema. En vez de ello, la descomposición se aplica en dos áreas principales: 1) la funcionalidad y el contenido (información) que deben entregarse y 2) el proceso que se usará para entregarlo.

Los seres humanos tienden a aplicar una estrategia de "divide y vencerás" cuando se enfrentan a un problema complejo. Dicho de manera simple, un problema complejo se divide en problemas más pequeños que son más manejables. Ésta es la estrategia que se aplica conforme comienza la planeación del proyecto. Las funciones del software, descritas en el enunciado del ámbito, se evalúan y refinan para proporcionar más detalle antes de comenzar la estimación (capítulo 26). Puesto que tanto las estimaciones de costo como las de calendario se orientan funcionalmente, con frecuencia es útil cierto grado de descomposición. De igual modo, los principales objetos de contenido o datos se descomponen en sus partes constituyentes, lo que proporciona una comprensión razonable de la información que se va a producir con el software.

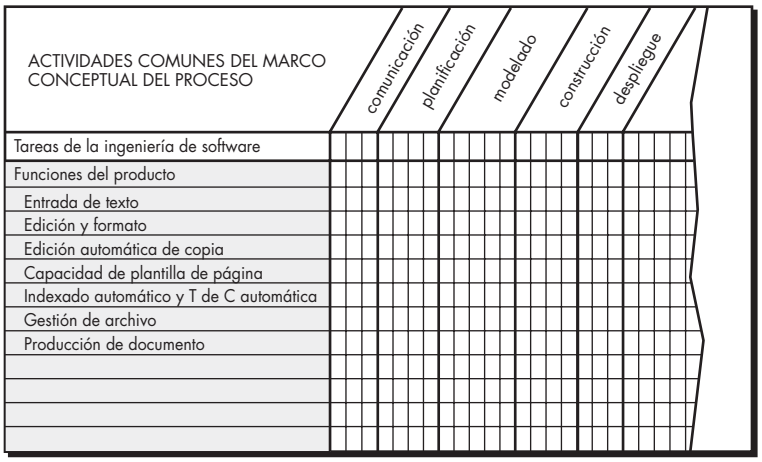
Tome como ejemplo un proyecto que construirá un nuevo producto de procesamiento de palabras. Entre las características únicas del producto, están la entrada continua de voz, así como un teclado virtual a través de una pantalla táctil, características extremadamente sofisticadas de "edición de copia automática", capacidad de plantilla de página, indexado automático y tabla de contenidos automática. El gerente del proyecto debe establecer primero un enunciado del ámbito que acote dichas características (y otras funciones más comunes, tales como edición, gestión de archivos y producción de documentos). Por ejemplo, ¿la entrada continua de voz requiere que el usuario "entrene" al producto? Específicamente: ¿qué capacidades ofrecerá el editor de copia? ¿Cuán sofisticadas serán las capacidades de la plantilla de página?, ¿éstas abarcarán las capacidades que requiere una pantalla táctil?

Conforme avanza la determinación de ámbito, ocurre de manera natural un primer nivel de división. El equipo de proyecto aprende que el departamento de mercadotecnia habló con los clientes potenciales y descubrió que las siguientes funciones deben ser parte de la edición automática de copia: 1) corrector de vocabulario, 2) corrector gramatical, 3) comprobación de referencias para documentos grandes (por ejemplo, ¿una referencia a una entrada bibliográfica se encuentra en la lista de entrada en la bibliografía?), 4) implementación de una característica de hoja de estilo que imponga consistencia a través de un documento y 5) validación de referencias de sección y capítulo para documentos grandes. Cada una de estas características representa una subfunción por implementar en el software. Cada una puede refinarse aún más si la descomposición hace más sencilla la planificación.

24.4 El proceso

Las actividades del marco conceptual (capítulo 2) que caracterizan al proceso de software son aplicables a todos los proyectos de software. El problema es seleccionar el modelo de proceso que sea adecuado para el software que el equipo del proyecto someterá a ingeniería.

FIGURA 24.1
Fusión de problema y proceso



El equipo debe decidir qué modelo de proceso es más adecuado: 1) para los clientes que solicitaron el producto y el personal que hará el trabajo, 2) para las características del producto en sí y 3) para el entorno de proyecto donde trabaja el equipo de software. Cuando se selecciona un modelo de proceso, el equipo define entonces un plan de proyecto preliminar con base en el conjunto de actividades del marco conceptual del proceso. Una vez establecido el plan preliminar, comienza la descomposición del proceso, es decir, debe crearse un plan completo que refleje las tareas laborales requeridas para poblar las actividades del marco conceptual. Estas actividades se exploran brevemente en las secciones que siguen; en el capítulo 26 se presenta una visión más detallada.

24.4.1 Fusión de producto y proceso

La planificación del proyecto comienza con la fusión de producto y proceso. Cada función que se va a someter a ingeniería por parte del equipo debe pasar a través del conjunto de actividades de marco conceptual que defina la organización de software.

Suponga que la organización adoptó las actividades genéricas del marco conceptual que se estudiaron en el capítulo 2: **comunicación**, **planificación**, **modelado**, **construcción** y **despliegue**. Los miembros del equipo que trabajen en una función del producto aplicarán en ella cada una de las actividades del marco conceptual. En esencia, se crea una matriz similar a la que se muestra en la figura 24.1. Cada función de producto principal (las funciones anotadas con números para el software de procesamiento de palabra estudiadas anteriormente) se menciona en la columna izquierda. Las actividades de marco conceptual se mencionan en la fila superior. Las tareas del trabajo de la ingeniería de software (para cada actividad del marco conceptual) ingresarán en la fila siguiente.[5] La labor del gerente de proyecto (y otros miembros del equipo) es estimar los requerimientos de recurso para cada celda de la matriz, fechas de inicio y término de las tareas asociadas con cada celda, y los productos operativos que se van a producir como consecuencia de cada tarea. Dichas actividades se consideran en el capítulo 26.

24.4.2 Descomposición del proceso

Un equipo de software debe tener un grado significativo de flexibilidad al elegir el modelo de proceso de software que es mejor para el proyecto y las tareas de la ingeniería de software que

5 Obsérvese que las tareas deben adaptarse a las necesidades específicas del proyecto, con base en algunos criterios de adaptación.

PUNTO CLAVE

El marco conceptual del proceso establece un esqueleto para la planificación del proyecto y se adapta para abarcar un conjunto de tareas que son adecuadas para el proyecto.

pueblen el modelo de proceso una vez elegido. Un proyecto relativamente pequeño que sea similar a esfuerzos anteriores puede lograrse mejor al usar el enfoque secuencial lineal. Si la fecha límite es tan apretada como para que toda la funcionalidad no pueda entregarse razonablemente, puede ser mejor una estrategia incremental. De igual modo, los proyectos con otras características (por ejemplo, requerimientos de incertidumbre, tecnología innovadora, clientes difíciles, significativo potencial de reuso) conducirán a la selección de otros modelos de proceso.[6]

Una vez elegido el modelo de proceso, el marco conceptual del proceso se adapta a él. En todo caso, puede usarse el marco conceptual genérico de proceso que se estudió anteriormente. Funcionará para los modelos lineales, para modelos iterativos e incrementales, para modelos evolutivos e incluso para modelos concurrentes o de ensamble de componentes. El marco conceptual del proceso es invariante y sirve como la base para todo el trabajo que realiza una organización de software.

Pero las tareas del trabajo real sí varían. La descomposición del proceso comienza cuando el gerente de proyecto pregunta: ¿cómo logramos esta actividad del marco conceptual? Por ejemplo, un proyecto simple y relativamente pequeño puede requerir las siguientes tareas para la actividad de comunicación:

1. Desarrollar lista de clarificación de conflictos.
2. Reunirse con los participantes para abordar la clarificación de conflictos.
3. Desarrollar en conjunto un enunciado del ámbito.
4. Revisar el enunciado del ámbito con todos los interesados.
5. Modificar el enunciado del ámbito según se requiera.

Estos eventos pueden ocurrir durante un periodo de menos de 48 horas. Representan una descomposición de proceso que es adecuada para el pequeño proyecto relativamente simple.

Ahora, considere un proyecto más complejo, que tenga un ámbito más amplio e impacto empresarial más significativo. Tal proyecto puede requerir las siguientes tareas para la **comunicación**:

1. Revisar la solicitud del cliente.
2. Planificar y calendarizar una reunión formal facilitada con todos los participantes.
3. Realizar investigación para especificar la solución propuesta y los enfoques existentes.
4. Preparar un "documento de trabajo" y una agenda para la reunión formal.
5. Realizar la reunión.
6. Desarrollar conjuntamente miniespecificaciones que reflejen las características de datos, funcionales y de comportamiento del software. De manera alternativa, desarrollar casos de uso que describan el software desde el punto de vista del usuario.
7. Revisar cada miniespecificación o usar casos de uso para ver su exactitud, consistencia y falta de ambigüedad.
8. Ensamblar las miniespecificaciones en un documento de ámbito.
9. Revisar el documento de ámbito o colección de casos de uso con todos los interesados.
10. Modificar el documento de ámbito o casos de uso según se requiera.

Ambos proyectos realizan la actividad de marco conceptual que se llama **comunicación**, pero el primer equipo de proyecto realiza la mitad de tareas de trabajo de ingeniería de software que el segundo.

---

6 Recuerde que las características del proyecto también tienen mucho apoyo en la estructura del equipo de software (sección 24.2.3).

24.5 El proyecto

Para administrar un proyecto de software exitoso, se debe comprender qué puede salir mal, de modo que los problemas puedan evitarse. En un excelente ensayo acerca de los proyectos de software, John Reel [Ree99] define 10 señales que indican que un proyecto de sistemas de información está en peligro:

¿Cuáles son las señales de que un proyecto de software está en peligro?

1. El personal del software no entiende las necesidades del cliente.
2. El ámbito del producto está pobremente definido.
3. Los cambios se gestionan pobremente.
4. Cambia la tecnología elegida.
5. Las necesidades empresariales cambian [o están mal definidas].
6. Las fechas límite son irreales.
7. Los usuarios son resistentes.
8. Pérdida de patrocinio [o nunca obtenido adecuadamente].
9. El equipo del proyecto carece de personal con habilidades adecuadas.
10. Los gerentes [y profesionales] evitan mejores prácticas y lecciones aprendidas.

Cita:

"No tenemos tiempo para detenernos por combustible, ya vamos retrasados."

M. Cleron

Los profesionales de la industria, hastiados, con frecuencia se refieren a la regla 90-90 cuando estudian proyectos de software particularmente difíciles: el primer 90 por ciento de un sistema absorbe el 90 por ciento del esfuerzo y tiempo asignados. El último 10 por ciento toma otro 90 por ciento del esfuerzo y tiempo asignados [Zah94]. Las semillas que conducen a la regla 90-90 están contenidas en las señales anotadas en la lista anterior.

Pero, ¡basta de negatividad! ¿Cómo actúa un gerente para evitar los problemas recién anotados? Reel [Ree99] sugiere un enfoque de sentido común de cinco partes en los proyectos de software:

1. *Comenzar con el pie derecho.* Esto se logra al trabajar duro (muy duro) para entender el problema que debe resolverse y luego establecer objetivos y expectativas realistas para todos aquellos que estarán involucrados en el proyecto. Lo anterior se refuerza al construir el equipo correcto (sección 24.2.3) y darle autonomía, autoridad y tecnología necesarias para realizar el trabajo.

Cita:

"Un proyecto es como un viaje en carretera. Algunos son simples y rutinarios, como conducir hacia la tienda a plena luz del día. Pero la mayoría de los proyectos que vale la pena realizar son más parecidos a conducir una camioneta 4×4 en las montañas y de noche."

Cem Kaner, James Bach y Bret Pettichord

2. *Mantener la cantidad de movimiento.* Muchos proyectos parten hacia un buen comienzo y luego lentamente se desintegran. A fin de mantener la cantidad de movimiento, el gerente de proyecto debe proporcionar incentivos para mantener la rotación de personal en un mínimo absoluto, el equipo debe enfatizar la calidad en cada tarea que realice y el administrador ejecutivo debe hacer todo lo posible para permanecer fuera del camino del equipo.[7]
3. *Siga la pista al progreso.* Para un proyecto de software, el progreso se rastrea conforme los productos operativos (por ejemplo, modelos, código fuente, conjuntos de casos de prueba) se producen y aprueban (usando revisiones técnicas) como parte de una actividad que asegure la calidad. Además, pueden recopilarse medidas de proceso de software y proyecto (capítulo 25) y usarse para valorar el progreso contra promedios desarrollados para la organización de desarrollo del software.

7 La implicación de esta afirmación es que la burocracia se reduce al mínimo, las reuniones extrañas se eliminan y se quita el énfasis a la adhesión dogmática a las reglas de proceso y proyecto. El equipo debe ser autoorganizado y autónomo.

4. *Tome decisiones inteligentes.* En esencia, las decisiones del gerente del proyecto y del equipo de software deben "mantenerse simples". Siempre que sea posible, decida usar software comercial de anaquel, o componentes o patrones de software existentes, así como evitar interfaces a la medida cuando estén disponibles enfoques estándar; decida también identificar y luego evitar los riesgos obvios, y asignar más tiempo del que se considere necesario para tareas complejas y riesgosas (necesitará cada minuto).
5. *Realice un análisis postmortem*. Establezca un mecanismo consistente para extraer lecciones aprendidas por cada proyecto. Evalúe los calendarios planeado y real, recopile y analice métricas de proyecto de software, consiga retroalimentación de los miembros del equipo y de los clientes, y registre los hallazgos en forma escrita.

24.6 El principio W^{5}HH

En un excelente ensayo acerca del proceso de software y los proyectos, Barry Boehm [Boe96] afirma: "necesita un principio de organización que reduzca la escala a fin de proporcionar planes [de proyecto] simples para proyectos simples". Boehm sugiere un enfoque que aborda los objetivos del proyecto, hitos y calendarios, responsabilidades, enfoques administrativos y técnicos, y recursos requeridos. Él lo llama *principio W^{5}HH*, por una serie de preguntas que conducen a una definición de las características clave del proyecto y al plan de proyecto resultante:

¿Cómo se definen las características clave del proyecto?

¿Por qué (why) *se desarrollará el sistema?* Todos los participantes deben valorar la validez de las razones empresariales para el trabajo de software. ¿El propósito de la empresa justifica el gasto de personal, tiempo y dinero?

¿Qué (what) *se hará?* Defina el conjunto de tareas requeridas para el proyecto.

¿Cuándo (when) *se hará?* El equipo establece un calendario de proyecto al identificar cuándo se realizarán las tareas del proyecto y cuándo se alcanzarán los hitos.

¿Quién (who) *es responsable de cada función?* Defina el papel y la responsabilidad de cada miembro del equipo de software.

¿Dónde (where) *se ubicarán en la organización?* No todos los roles y responsabilidades residen dentro de los profesionales del software. Clientes, usuarios y otros participantes también tienen responsabilidades.

¿Cómo (how) *se hará el trabajo, técnica y organizativamente?* Una vez establecido el ámbito del producto, debe definirse una estrategia técnica para el proyecto.

¿Cuánto (how much) *se necesita de cada recurso?* La respuesta a esta pregunta se deriva al desarrollar estimaciones (capítulo 26) con base en las respuestas a las preguntas anteriores.

El principio W^{5}HH de Boehm es aplicable sin importar el tamaño o complejidad de un proyecto de software. Las preguntas anotadas ofrecen un excelente esbozo de la planificación.

24.7 Prácticas cruciales

El Airlie Council[8] desarrolló una lista de "prácticas de software cruciales para administración basada en desempeño". Dichas prácticas "las usan consistentemente, y las consideran cruciales,

8 El Airlie Council incluyó un equipo de expertos en ingeniería de software contratados por el Departamento de Defensa estadounidense para ayudar a desarrollar lineamientos para mejores prácticas en administración de proyectos de software e ingeniería de software. Para conocer más acerca de mejores prácticas, vea **www.swqual.com/newsletter/vol1/no3/vol1no3.html**

proyectos y organizaciones enormemente exitosas cuya 'línea base' para el desempeño es consistentemente mucho mejor que el promedio industrial" [Air99].

Las prácticas cruciales[9] incluyen: administración del proyecto basada en métrica (capítulo 25), estimación empírica de costo y calendario (capítulos 26 y 27), rastreo del valor ganado (capítulo 27), rastreo de defecto contra metas de calidad (capítulos del 14 al 16) y administración consciente del personal (sección 24.2). Cada una de estas prácticas cruciales se aborda a lo largo de las partes 3 y 4 de este libro.

HERRAMIENTAS DE SOFTWARE



Herramientas de software para gerentes de proyecto

Las "herramientas" citadas aquí son genéricas y se aplican a un amplio rango de actividades realizadas por los gerentes de proyecto. En capítulos finales se consideran las herramientas específicas de administración de proyecto (por ejemplo, herramientas de calendarización, de estimación, de análisis de riesgo).

Herramientas representativas:[10]

El Software Program Manager's Network (**www.spmn.com**) desarrolló una herramienta simple llamada *Project Control Panel*, que brinda a los gerentes de proyecto un indicio directo del estado del proyecto. La herramienta tiene "calibradores" muy parecidos a un tablero y se implementa con Microsoft Excel. Está disponible para descarga en **www.spmn.com/products_software.html**

Gantthead.com (**www.gantthead.com/**) desarrolló un conjunto de útiles *listas de comprobación para gerentes de proyecto*.

Ittoolkit.com (**www.ittoolkit.com**) proporciona "una colección de guías de planificación, plantillas de proceso y hojas de trabajo inteligentes" disponible en CD-ROM.

24.8 RESUMEN

La administración de proyectos de software es una actividad sombrilla dentro de la ingeniería de software. Comienza antes de iniciar cualquier actividad técnica y continúa a lo largo del modelado, construcción y despliegue del software de cómputo.

Cuatro P tienen influencia sustancial sobre la administración del proyecto de software: personal, producto, proceso y proyecto. El personal debe organizarse en equipos eficaces, motivados para hacer trabajo de software de alta calidad, y coordinarse para lograr comunicación efectiva. Los requerimientos del producto deben comunicarse de cliente a desarrollador, dividirse (descomponerse) en sus partes constitutivas y ubicarse para su trabajo por parte del equipo de software. El proceso debe adaptarse al personal y al producto. Se selecciona un marco conceptual común al proceso, se aplica un paradigma de ingeniería de software adecuado y se elige un conjunto de tareas de trabajo para realizar el trabajo. Finalmente, el proyecto debe organizarse de forma que permita triunfar al equipo de software.

El elemento esencial en todos los proyectos de software es el personal. Los ingenieros del software pueden organizarse en diferentes estructuras de equipo que van desde las jerarquías tradicionales de control hasta los equipos de "paradigma abierto". Para apoyar el trabajo del equipo, pueden aplicarse varias técnicas de coordinación y comunicación. En general, las revisiones técnicas y la comunicación informal persona a persona tienen más valor para los profesionales.

La actividad de administración del proyecto abarca medición y métricas, estimación y calendarización, análisis de riesgos, rastreo y control. Cada uno de estos temas se considera en los capítulos siguientes.

---

9 Aquí sólo se mencionan aquellas prácticas cruciales asociadas con "integridad del proyecto".

10 Las herramientas que se mencionan aquí no representan un respaldo, sino una muestra de las herramientas en esta categoría. En la mayoría de los casos, los nombres de las herramientas son marcas registradas por sus respectivos desarrolladores.

Problemas y puntos por evaluar

24.1. Con base en la información contenida en este capítulo y en su propia experiencia, desarrolle "diez mandamientos" para empoderar a los ingenieros del software, es decir, elabore una lista de 10 lineamientos que conducirán al personal de software a que trabaje a toda su potencia.

24.2. El modelo de madurez de capacidades del personal (People-CMM) de The Software Engineering Institute, People-CMM echa un vistazo organizado a "áreas prácticas clave" que cultivan buen personal de software. Su instructor le asignará una APC para análisis y resumen.

24.3. Describa tres situaciones de la vida real en las que el cliente y el usuario final sean el mismo. Describa tres situaciones en las que sean diferentes.

24.4. Las decisiones tomadas por los administradores ejecutivos pueden tener un impacto significativo sobre la efectividad de un equipo de ingeniería del software. Proporcione cinco ejemplos para ilustrar que esto es cierto.

24.5. Revise el libro de Weinberg [Wei86] y escriba un resumen, con una extensión de dos a tres páginas, de los temas que deben considerarse al aplicar el modelo MOI.

24.6. Al lector se le asigna una gerencia de proyecto dentro de una organización de sistemas de información. Su labor será construir una aplicación que sea muy similar a otras que su equipo construyó, aunque ésta será más grande y más compleja. Los requerimientos se documentaron ampliamente por parte del cliente. ¿Qué estructura de equipo elegiría y por qué? ¿Qué modelo de proceso de software elegiría y por qué?

24.7. Al lector se le asigna una gerencia de proyecto para una pequeña compañía de productos de software. Su labor será construir un producto innovador que combine hardware de realidad virtual con software de última generación. Puesto que la competencia para el mismo mercado de entretenimiento es intensa, existe una presión significativa para tener listo el trabajo. ¿Qué estructura de equipo elegiría y por qué? ¿Qué modelo de proceso de software elegiría y por qué?

24.8. Al lector se le asigna una gerencia de proyecto para una gran compañía de productos de software. Su labor será administrar el desarrollo de la versión de siguiente generación de su software de procesamiento de palabras ampliamente usado. Puesto que la competencia es intensa, se establecieron y anunciaron apretadas fechas límite. ¿Qué estructura de equipo elegiría y por qué? ¿Qué modelo de proceso de software elegiría y por qué?

24.9. Al lector se le asigna una gerencia de proyecto de software para una compañía que atiende al mundo de la ingeniería genética. Su labor será administrar el desarrollo de un nuevo producto de software que acelerará el ritmo de tipificación genética. El trabajo está orientado a investigación y desarrollo, pero la meta es elaborar un producto dentro del próximo año. ¿Qué estructura de equipo elegiría y por qué? ¿Qué modelo de proceso de software elegiría y por qué?

24.10. Al lector se le ha pedido desarrollar una pequeña aplicación que analice cada curso ofrecido en la universidad y reporte las calificaciones promedio obtenidas en el curso (por un determinado periodo). Exponga el alcance y las limitaciones de este trabajo.

24.11. Haga una descomposición funcional de primer nivel de la función de plantilla de página que se estudió brevemente en la sección 24.3.2.

Lecturas y fuentes de información adicionales

El Project Management Institute (*Guide to the Project management body of Knowledge*, PMI, 2001) abarca todos los aspectos importantes de la administración de proyectos. Bechtold (*Essentials of Software Project Management,* 2a. ed., Management Concepts, 2007), Wysocki (*Effective Software Project Management,* Wiley, 2006), Stellman y Greene (*Applied Software Project Management,* O'Reilly, 2005), y Berkun (*The Art of Project Management,* O'Reilly, 2005) enseñan habilidades básicas y ofrecen lineamientos detallados para todas las tareas de administración de proyectos de software. McConnell (*Professional Software Development*, Addison-Wesley, 2004) ofrece consejo pragmático para lograr "calendarios más cortos, productos de mayor calidad y proyectos más exitosos". Henry (*Software Project Management*, Addison-Wesley, 2003) ofrece consejo del mundo real que es útil para todos los gerentes de proyecto.

Tom DeMarco *et al.* (*Adrenaline Junkies and Template Zombies*, Dorset House, 2008) escribieron un tratamiento comprensivo de los patrones humanos que se encuentran en todo proyecto de software. Una excelente serie de cuatro volúmenes, escrito por Weinberg (*Quality Software Management*, Dorset House, 1992, 1993, 1994, 1996), presenta conceptos de sistemas básicos de pensamiento y administración, explica cómo usar efectivamente las mediciones y aborda la "acción congruente", la habilidad para establecer "ajuste" entre las necesidades del gerente, las necesidades del personal técnico y las necesidades de la empresa. Ello proporcionará información útil a los gerentes novatos y a los expertos. Futrell *et al.* (*Quality Software Project Management*, Prentice-Hall, 2002) presentan un voluminoso tratamiento de la administración de proyectos. Brown *et al.* (*Antipatterns in Project Management*, Wiley, 2000) plantean qué no hacer durante la administración de un proyecto de software.

Brooks (*The Mythical Man-Month*, Anniversary Edition, Addison-Wesley, 1995) actualizó su libro clásico para ofrecer nueva comprensión de los temas de proyecto y administración de software. McConnell (*Software Project Survival Guide*, Microsoft Press, 1997) presenta excelente lineamiento pragmático para quienes deben administrar proyectos de software. Purba y Shah (*How to Manage a Successful Software Project*, 2a. ed., Wiley, 2000) presentan diversos estudios de caso que indican por qué algunos proyectos triunfan y otros fracasan. Bennatan (*On Time Within Budget*, 3a. ed., Wiley, 2000) presenta consejos útiles y lineamientos para gerentes de proyecto de software. Weigers (*Practical Project Initiation*, Microsoft Press, 2007) proporciona lineamientos prácticos para poner en marcha exitosamente un proyecto de software.

Puede argumentarse que el aspecto más importante de la administración del proyecto de software es la administración de personas. Cockburn (*Agile Software Development*, Addison-Wesley, 2002) presenta uno de los mejores análisis del personal de software escrito a la fecha. DeMarco y Lister [DeM98] escribieron el libro definitivo acerca del personal de software y los proyectos del software. Además, en años recientes se publicaron los siguientes libros acerca de la materia y vale la pena examinarlos:

Cantor, M., *Software Leadership: A Guide to Successful Software Development,* Addison-Wesley, 2001.

Carmel, E., *Global Software Teams: Collaborating Across Borders and Time Zones,* Prentice Hall, 1999.

Constantine, L., *Peopleware Papers: Notes on the Human Side of Software,* Prentice Hall, 2001.

Garton, C., y K. Wegryn, *Managing Without Walls,* McPress, 2006.

Humphrey, W. S., *Managing Technical People: Innovation, Teamwork, and the Software Process,* Addison-Wesley, 1997.

Humphrey, W. S., *TSP-Coaching Development Teams,* Addison-Wesley, 2006.

Jones, P. H., *Handbook of Team Design: A Practitioner's Guide to Team Systems Development,* McGraw-Hill, 1997.

Karolak, D. S., *Global Software Development: Managing Virtual Teams and Environments,* IEEE Computer Society, 1998.

Peters, L., *Getting Results from Software Development Teams,* Microsoft Press, 2008.

Whitehead, R., *Leading a Software Development Team,* Addison-Wesley, 2001.

Aun cuando no se relacionan específicamente con el mundo del software, y en ocasiones adolecen de sobresimplificación y gran generalización, los muy vendidos libros de "administración" de Kanter (*Confidence,* Three Rivers Press, 2006), Covy (*The 8th Habit,* Free Press, 2004), Bossidy (*Execution: The Discipline of Getting Things Done,* Crown Publishing, 2002), Drucker (*Management Challenges for the 21st Century,* Harper Business, 1999), Buckingham y Coffman (*First, Break All the Rules: What the World's Greatest Managers Do Differently,* Simon and Schuster, 1999), y Christensen (*The Innovator's Dilemma,* Harvard Business School Press, 1997) enfatizan "nuevas reglas" definidas por una economía rápidamente cambiante. Los títulos más antiguos, como *Who Moved My Cheese?* y *The One-Minute Manager* e *In Search of Excellence*, siguen ofreciendo valiosos elementos que pueden ayudar a administrar personal y proyectos de manera más efectiva.

En internet está disponible una gran variedad de fuentes de información acerca de las métricas de la administración de proyectos de software. Una lista actualizada de referencias existentes en la World Wide Web y que son relevantes para la administración de proyectos de software puede encontrarse en el sitio del libro: **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**